

27 - Prática: Modelos Generativos (III)

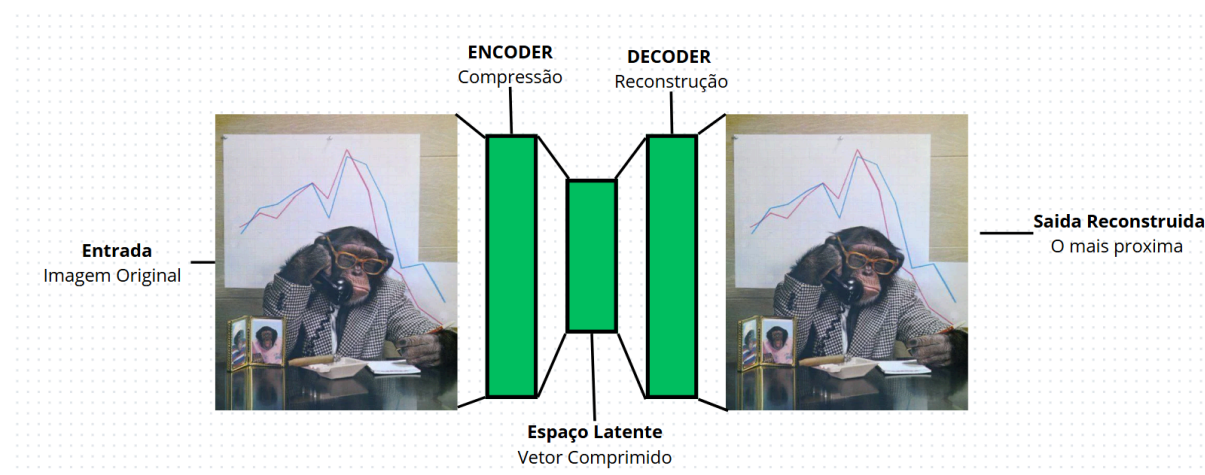
Ronny Gabryel Colatino de Souza

1 - Introdução

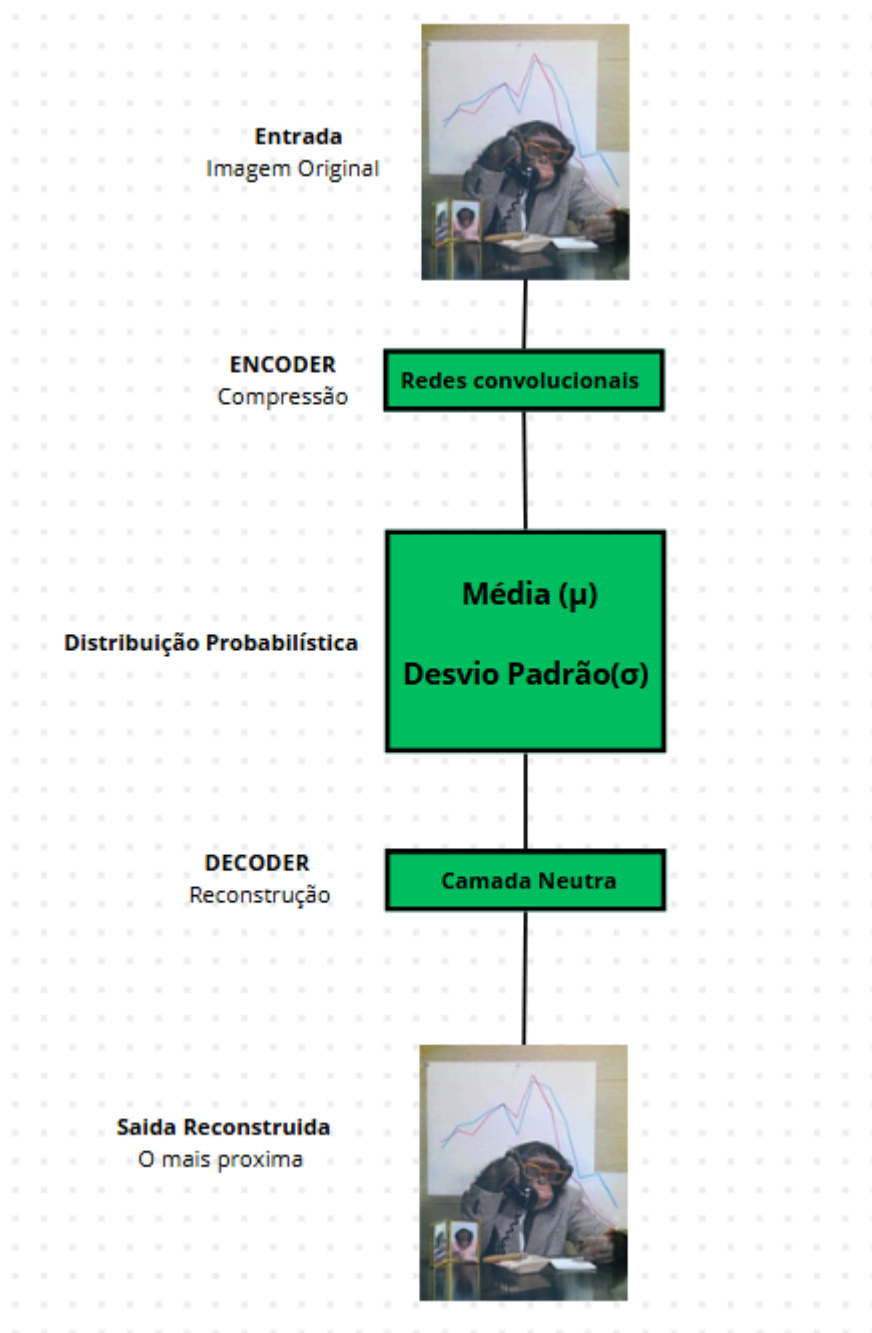
O objetivo desse card foi entender como funcionam os modelos generativos na prática, cobrindo desde os AutoEncoders tradicionais até os VAEs e GANs, e terminando com a construção de um GPT do zero treinado num corpus temático de World of Warcraft, o que tornou bem mais fácil ter intuição sobre o que o modelo estava aprendendo.

2 - Desenvolvimento

Os AutoEncoders são redes treinadas pra copiar a entrada na saída, mas de um jeito que força o modelo a aprender uma representação comprimida dos dados no meio do caminho, que é o espaço latente. O encoder comprime a entrada e o decoder reconstrói ela a partir dessa versão comprimida, e o objetivo é que a saída seja o mais parecida possível com a entrada original.



O VAE é uma evolução disso porque em vez de aprender um ponto fixo no espaço latente, ele aprende uma distribuição de probabilidade, o que permite gerar amostras novas que fazem sentido mesmo sem ter visto aquele dado específico antes. A métrica de erro do VAE tem duas partes o reconstruction loss, que penaliza imagens reconstruídas diferentes das originais, e o KL Divergence, que mede a distância entre a distribuição aprendida e uma distribuição normal padrão, forçando o espaço latente a ficar organizado o suficiente pra geração fazer sentido.



As GANs funcionam de um jeito completamente diferente porque em vez de uma rede única tentando reconstruir, existem duas redes competindo o gerador tenta criar dados falsos convincentes e o discriminador tenta distinguir o que é real do que é falso. O treinamento termina quando o discriminador não consegue mais diferenciar com precisão, ou seja, quando ele chega em cinquenta por cento de acerto, que é equivalente a chutar. O problema mais complicado das GANs é manter o balanceamento entre as duas redes, porque se uma delas ficar muito à frente da outra o treinamento desanda.

A parte prática foi a mais interessante porque em vez de usar as obras ou textos como nos exemplos da aula, usei um corpus de World of Warcraft com textos sobre

narrativa, MMORPGs e uso de LLMs em jogos, totalizando 22.042 caracteres e 2.961 palavras. O vocabulário ficou com 73 caracteres únicos, um pouco maior que o do Shakespeare por conta de caracteres especiais como parênteses, porcentagem e arroba que aparecem no corpus.

A tokenização foi feita por caractere, então cada letra, número ou símbolo vira um índice único, testei "Illidan" que retornou [31, 58, 55, 50, 47, 60] e de volta funcionou perfeitamente, o que confirma que o mapeamento estava certo. O corpus foi dividido em noventa por cento para treino 19.837 tokens, e os outros dez para validação 2.205 tokens.

A lógica do contexto e do alvo ficou bem clara visualizando os primeiros pares, onde dado o contexto "I" o alvo é "N", dado "IN" o alvo é "T", e assim por diante formando a palavra Introductio caractere por caractere. Isso mostra que o modelo aprende relações sequenciais entre os tokens, e quanto mais contexto ele tem, mais informação para fazer uma previsão boa.

Antes do treinamento a geração era completamente aleatória, saindo coisas como JnNVGME9Ta)ctBMWDRK@91PT7JF, que faz sentido porque o modelo ainda não aprendeu nada. Depois de 5.000 iterações com batch_size de 16, block_size de 64, 4 camadas e 4 cabeças de atenção, a loss de treino caiu de 2.07 para algo próximo de 1.26, mas a loss de validação ficou estagnada em torno de 2.7, o que indica overfitting claro dado o tamanho pequeno do corpus.

O resultado mais interessante foi o experimento com temperatura na geração. Com temperatura 0.7 o modelo ficou mais conservador e repetitivo, gerando fragmentos como narrative to content of the delitions offrom or also model maintailable narrative models, que tem palavras reconhecíveis mas ainda sem sentido. Com temperatura 1.0 saiu dynamic uning a stand generating pleaing, dialogue promined movid, e com 1.5 ficou mais caótico com Nachire quasing dyloing that balay shel-dorvenceificial. E deu para ver o quão controlado o modelo é na hora de escolher o próximo token, e dá para ver claramente como isso afeta a coerência do texto.

A arquitetura usada foi a do Transformer com as quatro classes principais o Head que implementa uma única cabeça de atenção com as projeções de query, key e value, a MultiHeadAttention que combina várias cabeças em paralelo para o modelo capturar diferentes tipos de relação ao mesmo tempo, a FeedForward que processa as representações depois da atenção, e o Block que junta tudo isso com LayerNorm e conexões residuais. O modelo final ficou com 212.809 parâmetros.

Uma detalhe que ficou bem claro no código é o uso da máscara triangular inferior, o Trill que impede o modelo de olhar para tokens futuros durante o treinamento. Sem isso o modelo colaria na resposta em vez de aprender a prever, o que tornaria a geração inútil na prática.

3 - Conclusão

Esse card foi bem legal porque pude usar o meu corpus de WoW que é tecnicamente mais denso, com vocabulário especializado de MMORPGs e LLMs, o que tornou o overfitting ainda mais evidente dado o tamanho pequeno do dataset. O que ficou mais claro foi a diferença que faz na geração e como o modelo sem contexto suficiente vai gerando fragmentos que lembram o tema mas sem coerência real. O mecanismo de atenção e a máscara triangular foram os conceitos que mais fizeram sentido na prática porque deu pra ver o efeito deles diretamente no que o modelo gerava.

4 - Referências

Videos do 27 - Prática: Modelos Generativos (III)

o git do Andrej Karpathy: <https://github.com/karpathy/nanoGPT>

Documentação: <https://blog.keras.io/building-autoencoders-in-keras.html>